Inductive Compliance: Lightweight & Noninvasive Transition-Gated Security for Declarative Infrastructure

Kavin Muthuselvan
Independent Researcher
k.muthuselvan@proton.me
https://github.com/Lycanthropy3301/Inductive-Compliance/

Abstract

Declarative infrastructure makes intended system state easier to describe, but not necessarily safer to update. Many modern compliance workflows determine whether a target configuration is acceptable by inspecting or transmitting more host state than is strictly necessary. In this talk, I propose a proof-of-concept model for noninvasive transition-gated security using NixOS as a base. Devices begin from compliant predecessor states and request updates in the form of context-aware atomic or guarded atomic transitions. Then, a local builder enforces authority-signed authorization for those specific `A -> B` changes. Within my talk, I will outline the threat model, demo a minimal finite-state prototype, and compare this approach to existing NixOS-based compliance gating systems. The ultimate goal is a lightweight, resilient, and privacy-preserving update coordination system for declarative systems. A model that is particularly useful where bandwidth, operator capacity, and infrastructure trust are constrained.

Proposal

Introduction

The words enforcement and noninvasive don't usually go hand in hand. Current fleet compliance systems often check target state snapshots. If current states are analyzed, they are done so through deep state analysis. This isn't the most egregious requirement. How can I trust a system if I don't keep track of its internals? However, these current methodologies have two main drawbacks:

1. They do not track unsafe predecessor state, and are hence blind to configuration drift and stale state.
2. Deep state analysis and telemetry is not only invasive, but very heavy on both system and network resources.

In the real world, certain updates can be unsafe relative to the current device state. For example, imagine a device part of a fleet system that requests a security downgrade in exchange for fewer privileges. During or after its transition to a less secure state, it could be compromised. If the system is then later upgraded to its original state, the newfound privileges in the context of an insecure predecessor state can be a recipe for backdoor disaster. A temporary move into a lower-security state may be justified operationally since it also reduces privilege or exposure. However, it can still create a compromise window whose consequences can persist when the system returns to its original high-risk, high-security state.

Declarative systems still need lineage-aware enforcement. Therefore, I propose a new system. Using NixOS as a base, we can focus less on verifying exact system state and instead project this concrete state into an abstract security configuration. In addition, let's not transmit state altogether, and instead focus on authorizing **state transitions** rather than final endpoint states. **This talk treats the update itself as a security object.**

The Process

Here is the basic step-by-step process of how Inductive Compliance works.

Assume a policy-compliant device in configuration state `A` wants to transition to configuration state `B`

1. Attestation server tracks abstract state `A`
2. Device sends a request to the Attestation Server with guarded atomic transition `T`
3. After validating T, the attestation server signs and sends a token `V` authorizing transition from state `A` to state `B`
4. Device begins build transition with token `V`. The authorized builder verifies the signed token `V`, checking that the predecessor state matches `A` and the successor state matches `B`
5. If checks pass, the build is successful and the device transitions from `A` to `B`

Now the important observation here is that the resultant state `B` is compliant, given that state `A` is compliant. The server in question does not need to manually attest to state `B`, it just needs to attest transition `T` and `B` remains compliant as a result of `A` and `T` being compliant. Repeating this process, as long as we start with a secure and attested base

image, we can ensure lineage-based policy compliance given that our policy is sufficiently defined through this system. Using this system, **compliance is enforced noninvasively over projected policy-relevant state.*

> The entire process will be demonstrated in the talk through a small NixOS proof-of-concept model in which a local builder verifies authority-signed authorization for specific state transitions. This proof-of-concept model is available here.

The Audience Takeaways and Relevance

This talk will be targeted at a universal audience, starting from the basics of declarative systems and configuration. **By the end of the talk, I aim to leave the audience with practical knowledge of**:

1. Configuring declarative systems with Nix/NixOS
2. The relationship between state and security (Configuration drift)
3. Noninvasive enforcement of security policies
4. How all three of these topics tie together with inductive compliance

NixOS is a fantastic model for declarative systems. In a world amidst an AI craze, state safety and compliance have become increasingly necessary with the increased speed in development and deployment of infrastructure. Having the knowledge of declarative systems, stateless security and compliance is incredibly useful in the modern age. Ultimately, I aim to educate the audience about these topics, using inductive compliance as a means to an end.

Due to the lightweight nature of Inductive Compliance, it presents itself as particularly useful for low-overhead infrastructure. Computing resources are getting scarce, and prices are being hiked up higher than ever before. Inductive compliance could be beneficial to both larger scale operations and community-scale systems due to its shift to projected security states. Additionally, there has been a constant struggle in computing between privacy and compliance. A noninvasive compliance control could have positive implications for consumer privacy for trust-critical infrastructure like banking applications and video game anti-cheat software.